Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
    - Explain Big O notation and how it helps in analyzing algorithms.
    - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
    - Create a class **Product** with attributes for searching, such as **productId, productName**, and **category**.
1. **Implementation:**
    - Implement linear search and binary search algorithms.
    - Store products in an array for linear search and a sorted array for binary search.
2. **Analysis:**
    - Compare the time complexity of linear and binary search algorithms.
    - Discuss which algorithm is more suitable for your platform and why.

Solution:

Big O notation is used to describe the time or space complexity of an algorithm as the input size (n) increases.

Best Case-

Linear Search- O(1) Binary Search- O(1)

Average Case-

Linear Search- O(n) Binary Search- O(logn)

Worst Case-

Linear Search- O(n) Binary Search- O(logn)

```
import java.util.*;
class Product {
   int id;
   String name;
   String type;
   Product(int id, String name, String type) {
      this.id = id;
      this.name = name;
      this.type = type;
   }
   void display() {
      System.out.println("Product ID   : " + id);
      System.out.println("Product Name : " + name);
      System.out.println("Category     : " + type);
   }
}
public class eCommerce {
   static Product linearSearch(Product[] items, int searchKey) {
      for (Product p : items) {
         if (p.id == searchKey) {
```

```
            return p;
        }
    }
    return null;
}
static Product binarySearch(Product[] items, int searchKey) {
    int left = 0;
    int right = items.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (items[mid].id == searchKey) {
            return items[mid];
        }

        if (items[mid].id < searchKey) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return null;
}

public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    System.out.println("E-Commerce Product Search");
```

```
System.out.print("Enter number of products: ");
int count = input.nextInt();
input.nextLine();
Product[] items = new Product[count];
for (int i = 0; i < count; i++) {
   System.out.println("\nProduct " + (i + 1));
   System.out.print("Enter Product ID: ");
   int pid = input.nextInt();
   input.nextLine();
   System.out.print("Enter Product Name: ");
   String pname = input.nextLine();
   System.out.print("Enter Category: ");
   String ptype = input.nextLine();
   items[i] = new Product(pid, pname, ptype);
}

System.out.print("\nEnter Product ID to Search: ");
int searchKey = input.nextInt();
Product result1 = linearSearch(items, searchKey);
System.out.println("\nLinear Search Result");
if (result1 != null) {
   result1.display();
} else {
   System.out.println("Product not found.");
}
Product[] sortedItems = Arrays.copyOf(items, items.length);
Arrays.sort(sortedItems, Comparator.comparingInt(p -> p.id));

Product result2 = binarySearch(sortedItems, searchKey);
```

```
        System.out.println("\nBinary Search Result");
        if (result2 != null) {
            result2.display();
        } else {
            System.out.println("Product not found.");
        }

        input.close();
    }
}
```

Output:

```
Enter Product ID to Search: 150

Linear Search Result
Product ID   : 150
Product Name : Watch
Category     : Accesories

Binary Search Result
Product ID   : 150
Product Name : Watch
Category     : Accesories
```

Analysis:

Binary Search is preferred because it is faster than Linear Search for large datasets. Linear Search takes O(n) time, while Binary Search takes O(log n) time. However, Binary Search requires the data to be sorted before searching.

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**
    - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
    - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
    - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**
    - Discuss the time complexity of your recursive algorithm.
    - Explain how to optimize the recursive solution to avoid excessive computation.

Solution:

Recursion:

Recursion is a method in which a function calls itself until a stopping condition is met.

```
import java.util.Scanner;
public class finForecasting {
   static double calculateAmount(double principal, double rate, int period) {
      if (period == 0) {
         return principal;
      }

      return calculateAmount(
            principal * (1 + rate / 100),
            rate,
```

```
            period - 1
        );
    }   public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.println("Financial Growth Calculator");

        System.out.print("Enter Initial Amount: ");
        double principal = input.nextDouble();

        System.out.print("Enter Interest Rate (%): ");
        double rate = input.nextDouble();

        System.out.print("Enter Number of Years: ");
        int period = input.nextInt();

        double result = calculateAmount(principal, rate, period);
        System.out.printf("\nAmount after %d years: %.2f",
            period,
            result);
        input.close();
    }
}
```

Output:

```
Financial Growth Calculator
Enter Initial Amount: 10000
Enter Interest Rate (%): 10
Enter Number of Years: 5

Amount after 5 years: 16105.10
```

Analysis:

The recurrence relation is T(n) = T(n-1) + O(1), where each recursive call performs a constant amount of work and reduces the problem size by 1. Solving the recurrence gives a time complexity of O(n).

The recursive solution can be optimized using iteration (loops) or dynamic programming/memoization. This avoids the overhead of multiple function calls and reduces memory usage. For this problem, a simple loop is the most efficient approach.